

show you how to use the `print()` function to get valuable debugging information from objects.

Sending Messages to Real-World Objects

Let's look at polymorphism in the real world, using cars as an example. All cars have an accelerator pedal. When the driver presses that pedal, they're sending the "accelerate" message to the car. The car they're driving could have an internal combustion engine or an electric motor, or be a hybrid. Each of these types of cars has its own implementation of what happens when it receives the accelerate message, and each behaves accordingly.

Polymorphism allows for easier adoption of new technology. If someone were to develop a nuclear-powered car, the user interface of the car would remain the same—the driver would still press the accelerator pedal to send the same message—but a very different mechanism would make the nuclear-powered car go faster.

As another real-world example, imagine you enter a large room with a bank of light switches that control a variety of different lights. Some of the bulbs are old-style incandescent bulbs, some are fluorescent, and some are newer LED bulbs. When you flip all the switches up, you are sending the "turn on" message to all the bulbs. The underlying mechanisms that cause incandescent, fluorescent, and LED bulbs to emit light are wildly different, but each achieves the user's intended goal.

A Classic Example of Polymorphism in Programming

In terms of OOP, polymorphism is about how client code can call a method with the exact same name in different objects, and each object will do whatever it needs to do to implement the meaning of that method for that object.

The classic example of polymorphism is to consider code that represents different types of pets. Let's say you have a collection of dogs, cats, and birds, and each understands some basic commands. If you ask these pets to